40

文件系统实现

在本章中，我们介绍一个简单的文件系统（file system）实现，叫作 **vsfs**（**Very Simple File System**，非常简单的文件系统）。它是典型 UNIX 文件系统的简化版，因此可以用来介绍一些基本的磁盘上的结构、访问方法（access method）和各种策略，这些在当今的许多文件系统中都能看到。

文件系统是纯软件；和前面讲 CPU 和内存虚拟化（virtualization）时不同，我们不会添加硬件特性来让文件系统的某个方面工作得更好（不过我们还是要留意设备的特性，确保文件系统运行良好）。正因为构建文件系统的灵活性很大，人们做出了许多各不相同的文件系统，简直是从 A 排到了 Z：从 AFS（Andrew File System）[H+88] 到 ZFS（Sun 的 Zettabyte File System）[B07]。这些文件系统各有各的数据结构（data structure），有些方面比同类强，有些方面则不如同类。因此，我们学习文件系统的方式是案例研究：本章先用一个简单的文件系统（vsfs）介绍大部分概念，然后研究一系列真实的文件系统，了解它们在实践中有哪些不同。

关键问题：如何实现一个简单的文件系统
如何构建一个简单的文件系统？磁盘上需要哪些结构？它们需要记录什么？如何访问它们？

40.1 思考的方式

思考文件系统时，我们通常建议从两个方面入手；如果这两个方面你都理解了，大概也就理解了文件系统的基本工作原理。

第一个方面是文件系统的**数据结构**（data structures）。换句话说，文件系统用哪些类型的磁盘上的结构来组织它的数据和元数据（metadata）？我们将看到的第一批文件系统（包括下面的 vsfs）采用简单的结构，比如块（block）或其他对象的数组；而更复杂的文件系统，比如 SGI 的 XFS，则使用更复杂的基于树的结构 [S+96]。

文件系统的第二个方面是它的**访问方法**（access methods）。它如何把进程发出的调用，比如 `open()`、`read()`、`write()` 等，映射到它的结构上？执行某个系统调用（system call）时，要读哪些结构？要写哪些结构？所有这些步骤执行得有多高效？

补充：文件系统的心智模型

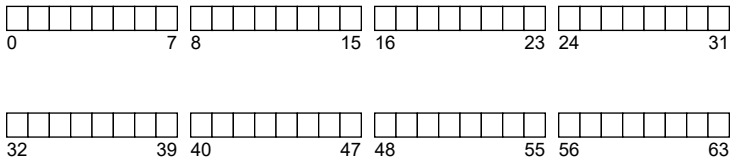
我们之前讨论过，学习系统时，你真正要建立的是心智模型。对文件系统来说，你的心智模型最终应该能回答这样一些问题：磁盘上的哪些结构存放着文件系统的数据和元数据？进程（process）打开一个文件时会发生什么？读或写的过程中会访问磁盘上的哪些结构？不断建立和改进心智模型，你就会对正在发生的事情形成抽象的理解，而不只是试图弄懂某段文件系统代码的细节（当然，弄懂代码也很有用！）。

如果你理解了文件系统的数据结构和访问方法（access method），就对它真正的工作方式有了一个很好的心智模型，这是系统思维的关键一环。下面深入研究第一个实现时，请试着逐步建立起这样的模型。

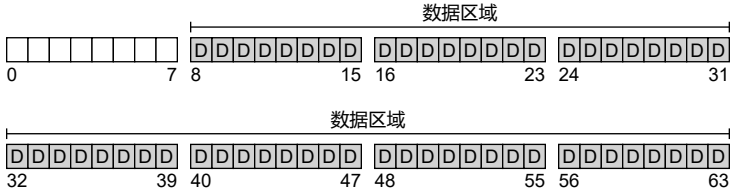
40.2 整体组织

现在我们来设计 vsfs 的数据结构在磁盘上的整体组织。首先要做的是把磁盘划分成**块**（block）；简单的文件系统只使用一种块大小（block size），我们这里也这样做，选用常见的 4 KB。

因此，在我们看来，要在上面构建文件系统的这个磁盘分区很简单：一串块，每块 4 KB。在一个大小为 N 个 4 KB 块的分区中，块的地址从 0 到 $N-1$。假设我们有一个很小的磁盘，只有 64 个块：



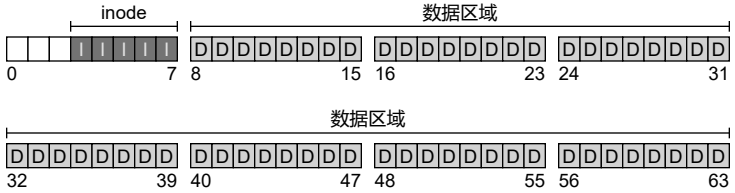
现在我们来想想，要构建一个文件系统，需要在这些块里存放什么。当然，首先想到的是用户数据。事实上，任何文件系统的大部分空间都是（而且应该是）用户数据。我们把磁盘上存放用户数据的区域叫作**数据区域**（data region），同样为了简单，在磁盘上为这些块预留一块固定的部分，比如 64 个块中的最后 56 个：



我们在上一章（稍微）了解过，文件系统必须记录每个文件的信息。这些信息是一种关键的**元数据**（metadata），记录着文件由数据区域（data region）中的哪些数据块（data block）

组成、文件的大小、它的所有者和访问权限、访问和修改时间等。为了存放这些信息，文件系统通常有一种叫作 **inode** 的结构（下面会进一步介绍 inode）。

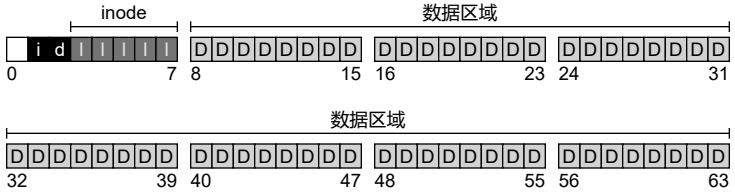
为了容纳 inode，我们也需要在磁盘上为它们预留一些空间。我们把这部分磁盘叫作 **inode 表**（inode table），它存放的就是一个磁盘上的 inode 数组。假设我们用 64 个块中的 5 个存放 inode（图中用 I 表示），磁盘映像现在是这个样子：



这里要指出，inode 通常不太大，比如 128 或 256 字节。假设每个 inode 256 字节，一个 4 KB 的块就能容纳 16 个 inode，上面的文件系统一共有 80 个 inode。我们这个简单文件系统建在只有 64 个块的小分区上，80 就是它最多能容纳的文件数；不过要注意，同样的文件系统如果构建在更大的磁盘上，只需分配一个更大的 inode 表，就能容纳更多文件。

到目前为止，我们的文件系统有了数据块（D）和 inode（I），但还缺几样东西。你可能已经猜到，还缺的一个主要部分，是用某种方法记录 inode 和数据块是空闲还是已分配。因此，这种**分配结构**（allocation structures）是任何文件系统都必不可少的组成部分。

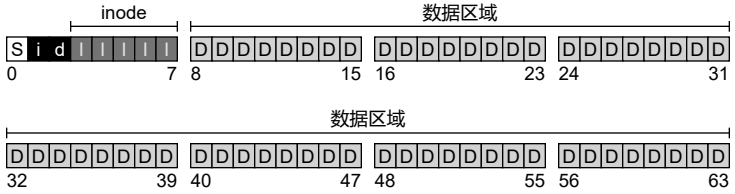
当然，记录分配情况的方法有很多。例如，我们可以用一个**空闲链表**（free list），它指向第一个空闲块（free block），第一个空闲块又指向下一个空闲块，依此类推。我们这里选择一种简单而流行的结构，叫作**位图**（bitmap），数据区域（data region）和 inode 表各用一个，分别叫**数据位图**（data bitmap）和 **inode 位图**（inode bitmap）。位图是一种简单的结构：每一位表示对应的对象或块是空闲（0）还是在用（1）。于是，加上 inode 位图（i）和数据位图（d）之后，新的磁盘布局如下：



你可能会注意到，用整整一个 4 KB 的块来存放这些位图有点大材小用；这样一个位图可以记录 32K 个对象是否已分配，而我们只有 80 个 inode 和 56 个数据块。不过，为了简单，我们还是给每个位图都分配一整个 4 KB 的块。

细心的读者（也就是还没睡着的读者）可能已经注意到，我们这个非常简单的文件系统，在设计磁盘结构时还剩下一个块没有安排。我们把它留给**超级块**（superblock），在下图中用 S 表示。超级块包含这个文件系统的信息，例如文件系统中有多少个 inode 和数据块（在这个例子中分别是 80 和 56）、inode 表从哪里开始（块 3）等等。它很可能还包含某种幻数（magic number），用来标识文件系统的类型（这里是 vsfs）。

 © 2008–26, ARPACI-DUSSEAU

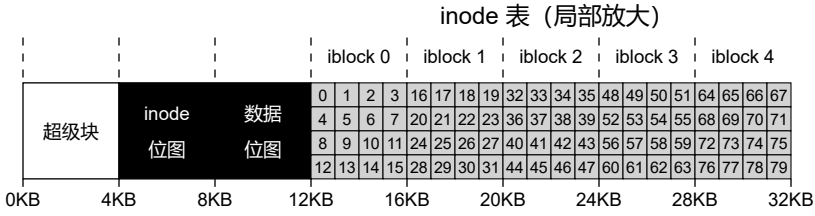


因此，挂载（mount）文件系统时，操作系统会先读取超级块，初始化各种参数，然后把这个卷（volume）挂接到文件系统树上。这样，访问卷中的文件时，系统就能确切地知道去哪里找所需的磁盘上的结构。

40.3 文件组织：inode

inode 是文件系统中最重要的磁盘上的结构之一；几乎所有文件系统都有类似的结构。inode 这个名字是 **index node**（**索引节点**）的简称，这是 UNIX [RT74]（也许还有更早的系统）里的老叫法。这个名字反映了一个事实：这些节点最初存放在一个数组中，系统通过索引这个数组来访问某个特定的 inode。

每个 inode 都隐式地对应一个数字（叫作 **inode 号**，i-number），我们之前把它叫作文件的 **low-level name**。在 vsfs 以及其他简单的文件系统中，给定一个 inode 号，就应该能直接算出对应的 inode 在磁盘上的位置。例如，拿上面 vsfs 的 inode 表来说：它的大小为 20 KB（5 个 4 KB 的块），因此包含 80 个 inode（假设每个 inode 256 字节）；再假设 inode 区域从 12 KB 开始（也就是说，超级块从 0 KB 开始，inode 位图在地址 4 KB 处，数据位图在 8 KB 处，inode 表紧随其后）。于是，在 vsfs 中，文件系统分区开头部分的布局如下（局部放大）：



补充：数据结构——inode

inode 是许多文件系统都在用的通用名称，指存放某个文件的元数据的结构，比如文件的长度、权限以及组成它的各个块的位置。这个名字至少可以追溯到 UNIX（很可能还能追溯到 Multics，甚至更早的系统）；它是 **index node** 的简称，因为要用 inode 号（inode number）作为索引，在磁盘上的 inode 数组中找到该编号的 inode。我们会看到，inode 的设计是文件系统设计的一个关键部分。大多数现代系统都给自己管理的每个文件配了一个这样的结构，只是叫法可能不同（比如 dnode、fnode 等）。

要读取 32 号 inode，文件系统首先计算它在 inode 区域内的偏移量（offset），即 $32 \cdot sizeof(inode) = 8192$，再加上磁盘上 inode 表的起始地址（`inodeStartAddr = 12KB`），就得到了所需 inode 块（inode block）的正确字节地址：20 KB。回想一下，磁盘不能按字节寻址，而是由大量可寻址的扇区（sector）组成，扇区通常是 512 字节。因此，为了取出包含 32 号 inode 的 inode 块（inode block），文件系统会向扇区 $\frac{20 \times 1024}{512}$，也就是 40 号扇区，发出读请求。更一般地，inode 块（inode block）所在的扇区地址 `sector` 可以这样计算：

```
blk    = (inumber * sizeof(inode_t)) / blockSize;
sector = ((blk * blockSize) + inodeStartAddr) / sectorSize;
```

关于一个文件，你需要的信息几乎都在它的 inode 里：它的*类型*（例如普通文件、目录（directory）等）、*大小*、*分配给它的块数*、*保护信息*（比如文件归谁所有、谁可以访问它）、*一些时间信息*，包括文件的创建、修改或最后访问时间，以及它的数据块在磁盘上的位置信息（比如某种指针）。我们把关于一个文件的所有这类信息称为**元数据**（metadata）；其实，文件系统里凡是不属于纯用户数据的信息，通常都这样称呼。图 40.1 是 ext2 [P09] 中的一个 inode 示例[1]。

inode 设计中最重要的决策之一，是它如何指向数据块所在的位置。一种简单的方法是在 inode 中放一个或多个**直接指针**（direct pointer，即磁盘地址）；每个指针指向属于该文

大小	名称	这个 inode 字段有什么用?
2	mode	这个文件能否被读、写、执行?
2	uid	这个文件归谁所有?
4	size	这个文件有多少字节?
4	time	这个文件最后一次被访问是什么时候?
4	ctime	这个文件的 inode 最后一次被修改是什么时候?
4	mtime	这个文件最后一次被修改是什么时候?
4	dtime	这个 inode 是什么时候被删除的?
2	gid	这个文件属于哪个组?
2	links_count	这个文件有多少个硬链接?
4	blocks	已经给这个文件分配了多少个块?
4	flags	ext2 应该如何使用这个 inode?
4	osd1	一个与操作系统相关的字段
60	block	一组磁盘指针（共 15 个）
4	generation	文件版本（供 NFS 使用）
4	file_acl	一种比 mode 位更新的权限模型
4	dir_acl	称为访问控制列表

图 40.1：**简化的 Ext2 inode**

[1]类型信息保存在目录项（directory entry）中，因此 inode 本身里没有这项信息。

 © 2008–26, ARPACI-DUSSEAU

提示：考虑基于区段的方法

另一种方法是用**区段**（extent）代替指针。一个区段就是一个磁盘指针加上一个长度（以块为单位）；因此，不需要为文件的每个块都准备一个指针，只需一个指针和一个长度，就能指明文件在磁盘上的位置。只用一个区段会有局限，因为分配文件时可能很难在磁盘上找到一块连续的空闲空间（free space）。因此，基于区段的文件系统通常允许使用多个区段，这样在分配文件时文件系统就有了更大的自由。

比较这两种方法，基于指针的方法更灵活，但每个文件要用大量元数据，大文件尤其如此。基于区段的方法没那么灵活，但更紧凑；特别是当磁盘上有足够的空闲空间、文件可以连续存放时（这本来就是几乎所有文件分配策略的目标），它们的效果很好。

件的一个磁盘块（disk block）。这种方法有局限：例如，如果你想要一个非常大的文件（比如比块大小乘以 inode 中直接指针的个数还大），那就没办法了。

多级索引

为了支持更大的文件，文件系统设计者不得不在 inode 中引入不同的结构。一个常见的思路是使用一种叫作**间接指针**（indirect pointer）的特殊指针。它不指向存放用户数据的块，而是指向一个存放更多指针的块，这些指针再分别指向用户数据。因此，一个 inode 可以有固定数量的直接指针（direct pointer），比如 12 个，再加一个间接指针（indirect pointer）。如果文件变得足够大，就（从磁盘的数据块区域）分配一个间接块（indirect block），并让 inode 中存放间接指针（indirect pointer）的那一项指向它。假设块大小为 4 KB、磁盘地址（disk address）为 4 字节，这样又多了 1024 个指针；文件最大可以达到 $(12 + 1024) \cdot 4K$，即 4144 KB。

不难想到，有了这种方法，你可能还想支持更大的文件。为此，只需在 inode 中再加一个指针：**二级间接指针**（double indirect pointer）。这个指针指向一个块，块中存放的是指向间接块（indirect block）的指针，每个间接块里又存放着指向数据块的指针。因此，一个二级间接块（double indirect block）让文件还能再增加 1024×1024 个，也就是 100 万个 4 KB 的块，换句话说，可以支持超过 4 GB 的文件。不过你可能还想要更大的，我们打赌你已经知道接下来是什么了：**三级间接指针**（triple indirect pointer）。

总的来说，用这种不平衡的树指向文件块的方法，叫作**多级索引**（multi-level index）。我们来看一个例子：有 12 个直接指针（direct pointer），以及一个一级间接块（indirect block）和一个二级间接块（double indirect block）。假设块大小为 4 KB、指针为 4 字节，这种结构可以容纳略大于 4 GB 的文件（即 $(12 + 1024 + 1024^2) \times 4$ KB）。你能算出再加上一个三级间接块（triple indirect block）后，能支持多大的文件吗？（提示：相当大）

许多文件系统都使用多级索引（multi-level index），包括常用的 Linux ext2 [P09] 和 ext3、NetApp 的 WAFL，以及最初的 UNIX 文件系统。另一些文件系统，包括 SGI XFS 和 Linux ext4，则使用**区段**（extent）而不是简单的指针；基于区段的方案如何工作，详见前面的提示框；区段有点像讨论虚拟内存（virtual memory）时讲过的段（segment）。

大多数文件都很小	最常见的大小约为 2K
平均文件大小在增长	平均大小接近 200K
大部分字节存放在大文件中	少数大文件占用了大部分空间
文件系统包含大量文件	平均接近 10 万个
文件系统大约是半满的	即使磁盘不断变大，文件系统仍保持约 50% 的占用率
目录通常很小	很多目录只有几个条目；大多数目录的条目不超过 20 个

图 40.2：**文件系统测量结果汇总**

你可能会想：为什么要用这样一棵不平衡的树？为什么不用别的方法？原来，许多研究人员考察过文件系统及其使用方式，几乎每一次都会发现一些几十年来始终成立的“真理”。其中一个发现是大多数文件都很小。这种不平衡的设计正反映了这一现实；如果大多数文件确实很小，就应该针对这种情况做优化。因此，只用少量的直接指针（12 是一个典型的数目），inode 就可以直接指向 48 KB 的数据，更大的文件才需要一个（或多个）间接块（indirect block）。比较新的一项研究见 Agrawal 等人的论文 [A+07]；图 40.2 总结了其中的结果。

当然，inode 的设计还有许多其他可能；毕竟，inode 只是一种数据结构，任何能存放相关信息、并能高效查询这些信息的数据结构都可以。文件系统软件很容易修改，所以如果工作负载（workload）或技术发生变化，你应该乐于探索不同的设计。

40.4 目录组织

在 vsfs 中（和许多文件系统一样），目录的组织很简单；一个目录基本上只包含一个由（条目名，inode 号）对组成的列表。目录里的每个文件或子目录，在该目录的数据块中都对应一个字符串和一个数字；每个字符串可能还带一个长度（假设名字长度可变）。

例如，假设目录 `dir` 的 inode 号为 5，其中有三个文件（`foo`、`bar` 和 `foobar_is_a_pretty_longname`），它们的 inode 号分别是 12、13 和 24。`dir` 在磁盘上的数据可能是这样的：

```
inum | reclen | strlen | name
   5      12       2      .
   2      12       3      ..
  12      12       4      foo
  13      12       4      bar
  24      36      28      foobar_is_a_pretty_longname
```

在这个例子中，每个条目都有一个 inode 号、记录长度（名字的总字节数加上剩余的空间）、字符串长度（名字的实际长度），最后是条目的名字。注意，每个目录都有两个额外的条目：`.`（“点”）和 `..`（“点点”）；点目录（dot directory）就是当前目录，在这个例子中是 `dir`；点点则是父目录，在这个例子中是根目录（root directory）。

补充：基于链接的方法

设计 inode 还有一种更简单的方法，就是使用**链表**（linked list）。这样，inode 内部不需要多个指针，只需要一个，指向文件的第一个块。文件更大时，就在这个数据块的末尾再放一个指针，依此类推，这样就能支持大文件了。

你可能已经猜到，链接方式的文件分配在某些工作负载下表现很差；比如想想读取文件的最后一个块，或者只是做随机访问（random access）。因此，为了让链接分配更好用，有些系统会在内存中保存一张链接信息表，而不是把 next 指针和数据块本身存在一起。这张表以数据块 D 的地址为索引；表项的内容就是 D 的 next 指针，即文件中紧跟在 D 后面的那个块的地址。表项里也可以是空值（表示文件结束），或者其他某种标记，表示某个块是空闲的。有了这样一张 next 指针表，链接分配方案就能高效地支持随机访问文件：先在（内存中的）表里扫描，找到所需的块，然后直接（在磁盘上）访问它。

这样的表听起来是不是很熟悉? 我们所描述的，正是**文件分配表**（file allocation table），也就是 **FAT** 文件系统的基本结构。没错，这个在 NTFS [C94] 之前的经典 Windows 文件系统，用的正是一种简单的链接分配方案。它和标准的 UNIX 文件系统还有其他区别；例如，它本身没有 inode，而是由目录项存放文件的元数据，并直接指向文件的第一个块，所以它没法创建硬链接（hard link）。更多不那么优雅的细节，见 Brouwer 的文章 [B02]。

删除一个文件（比如调用 `unlink()`）可能会在目录中间留下一段空白，因此也应该有某种方法把它标记出来（比如用一个保留的 inode 号，如零）。这种删除正是需要记录长度的原因之一：新条目可能会重用一个旧的、更大的条目，里面就会留有多余的空间。

你可能想知道目录到底存放在哪里。通常，文件系统把目录当作一种特殊类型的文件。因此，目录有一个 inode，位于 inode 表中的某处（inode 的类型字段标记为“目录”而不是“普通文件”）。inode 指向目录的数据块，可能还有间接块（indirect block）；这些块位于我们这个简单文件系统的数据块区域中。因此，我们的磁盘结构保持不变。

我们还要再次指出，这种简单的目录项线性列表并不是存放这类信息的唯一方式。和前面一样，任何数据结构都可以。例如，XFS [S+96] 把目录存成 B 树（B-tree）。创建文件时必须先确认文件名还没被用过，所以和使用简单列表、必须完整扫描列表的系统相比，XFS 创建文件更快。

补充：空闲空间管理

管理空闲空间的方法有很多；位图只是其中一种。一些早期的文件系统使用**空闲链表**（free list），超级块中保存一个指针，指向第一个空闲块；该块中又保存下一个空闲块的指针，这样就在系统的空闲块之间串起了一个链表。需要一个块时，就使用表头的块，并相应地更新链表。

现代文件系统使用更复杂的数据结构。例如，SGI 的 XFS [S+96] 使用某种形式的 **B 树**（B-tree）来紧凑地表示磁盘的哪些部分是空闲的。和任何数据结构一样，可以做出不同的时空权衡（time-space trade-off）。

40.5 空闲空间管理

文件系统必须记录哪些 inode 和数据块是空闲的、哪些不是，这样在分配新文件或目录时，才能为它找到空间。因此，**空闲空间管理**（free space management）对所有文件系统都很重要。在 vsfs 中，我们用两个简单的位图来完成这项任务。

例如，创建一个文件时，我们必须为它分配一个 inode。文件系统会在位图中找一个空闲的 inode 分配给这个文件，把它标记为已用（置为 1），并最终把正确的信息写回磁盘上的位图。分配数据块时也要做一组类似的操作。

为新文件分配数据块时，可能还要考虑其他一些因素。例如，ext2、ext3 等 Linux 文件系统在创建新文件并需要数据块时，会寻找一串连续的空闲块（比如 8 个）；找到这样一串空闲块分配给新文件，文件系统就能保证文件至少有一部分在磁盘上是连续的，从而提高性能。因此，这种**预分配**（pre-allocation）策略是为数据块分配空间时常用的启发式方法。

40.6 访问路径：读和写

现在我们对文件和目录在磁盘上如何存放有了一些概念，应该能够跟踪读或写一个文件时的操作流程了。因此，理解在这条**访问路径**（access path）上发生了什么，是理解文件系统工作原理的第二个关键；注意了！

在下面的例子中，我们假设文件系统已经挂载，因此超级块已经在内存中。其他所有东西（即 inode、目录）都还在磁盘上。

从磁盘读取文件

在这个简单的例子中，我们先假设你只是想打开一个文件（比如 `/foo/bar`），读取它，然后关闭它。为简单起见，假设这个文件只有 12 KB（即 3 个块）。

	数据位图	inode 位图	根目录 inode	foo inode	bar inode	根目录数据	foo 数据	bar 数据 [0]	bar 数据 [1]	bar 数据 [2]
open(bar)			**读**			**读**				
				读			**读**			
					读					
read()					**读**			**读**		
					写					
read()					**读**				**读**	
					写					
read()					**读**					**读**
					写					

图 40.3：**读取文件的时间线（时间向下递增）**

当你发出 `open("/foo/bar", O_RDONLY)` 调用时，文件系统首先要找到文件 `bar` 的 inode，以获得关于这个文件的一些基本信息（权限信息、文件大小等）。可文件系统手里只有完整的路径名（pathname），所以必须**遍历**（traverse）路径名，才能找到这个 inode。

所有遍历都从文件系统的根开始，也就是**根目录**（root directory），它就叫 `/`。因此，文件系统要从磁盘读取的第一样东西，就是根目录的 inode。但这个 inode 在哪里呢？要找到一个 inode，我们必须知道它的 inode 号。通常，一个文件或目录的 inode 号是在它的父目录中找到的；而根目录按定义没有父目录。因此，根目录的 inode 号必须是“众所周知”的，文件系统在挂载时就得知道它是多少。在大多数 UNIX 文件系统中，根目录的 inode 号是 2。因此，为了开始这个过程，文件系统读入包含 2 号 inode 的块，也就是第一个 inode 块（inode block）。

inode 读入后，文件系统就可以从中找到指向数据块的指针，根目录的内容就存放在这些数据块里。于是，文件系统用这些磁盘上的指针读取目录，在这个例子中是寻找 `foo` 的条目。读入一个或多个目录数据块后，它会找到 `foo` 的条目，从而知道 `foo` 的 inode 号（假设是 44），下一步要用到它。

下一步是递归地遍历路径名，直到找到所需的 inode。在这个例子中，文件系统读取包含 `foo` 的 inode 的块，再读取它的目录数据，最终找到 `bar` 的 inode 号。`open()` 的最后一步是把 `bar` 的 inode 读入内存；然后文件系统做最后的权限检查，在这个进程的打开文件表（open file table）中分配一个文件描述符（file descriptor），并把它返回给用户。

打开之后，程序就可以发出 `read()` 系统调用来读这个文件。第一次读（除非调用过 `lseek()`，否则从偏移量 0 开始）会读入文件的第一个块，读之前要查阅 inode 来找到这个块的位置；它可能还会用新的最后访问时间更新 inode。读操作还会更新内存中打开文件表里这个文件描述符的表项，修改其中的文件偏移量（file offset），这样下一次读就会读入文件的第二个块，依此类推。

到某个时候，程序会关闭文件。这里要做的工作少得多；显然，要释放文件描述符，但目前来说，文件系统真正要做的也就这些了。不会发生任何磁盘 I/O。

图 40.3（第 9 页）描绘了整个过程；图中时间从上往下递增。在图中，open 引发了多次读，才最终找到文件的 inode。之后，读取每个块时，文件系统都要先查阅 inode，然后读取这个块，再写一次 inode，更新它的最后访问时间字段。花点时间，弄明白这是怎么回事。

还要注意，open 产生的 I/O 量与路径名的长度成正比。路径中每多一层目录，我们就要多读它的 inode 和数据。如果目录很大，情况会更糟；在这里，我们只需读一个块就能得到目录的内容，而如果目录很大，可能要读很多个数据块才能找到所需的条目。没错，读

补充：读操作不访问分配结构

我们见过很多学生被位图之类的分配结构（allocation structure）弄糊涂。特别是，很多人以为，只是读取文件、不分配任何新块的时候，仍然要查阅位图。不是这样的！位图之类的分配结构（allocation structure）只有在需要分配时才会用到。inode、目录和间接块（indirect block）中有完成读请求所需的全部信息；inode 已经指向某个块了，就不需要再确认这个块是否已分配。

一个文件时，情况就已经相当糟糕了；你马上就会发现，写一个文件（尤其是创建一个新文件）更糟。

向磁盘写入文件

写文件的过程与此类似。首先，必须打开文件（如上所述）。然后，应用程序可以发出 `write()` 调用，用新内容更新文件。最后，关闭文件。

和读不同，写文件还可能需要**分配**（allocate）一个块（除非是在覆盖已有的块）。写一个新文件时，每次写操作不仅要把数据写入磁盘，还要先决定把哪个块分配给这个文件，并相应地更新磁盘上的其他结构，比如数据位图（data bitmap）和 inode。因此，每次写文件在逻辑上会产生五次 I/O：一次读数据位图（然后更新它，把新分配的块标记为已用），一次写位图（把它的新状态反映到磁盘上），再读写 inode 各一次（用新块的位置更新它），最后一次写实际的数据块本身。

如果考虑像创建文件这样简单而常见的操作，写流量就更大了。要创建一个文件，文件系统不仅要分配一个 inode，还要在包含新文件的目录中分配空间。为此产生的 I/O 流量相当大：读一次 inode 位图（找一个空闲的 inode），写一次 inode 位图（把它标记为已分配），

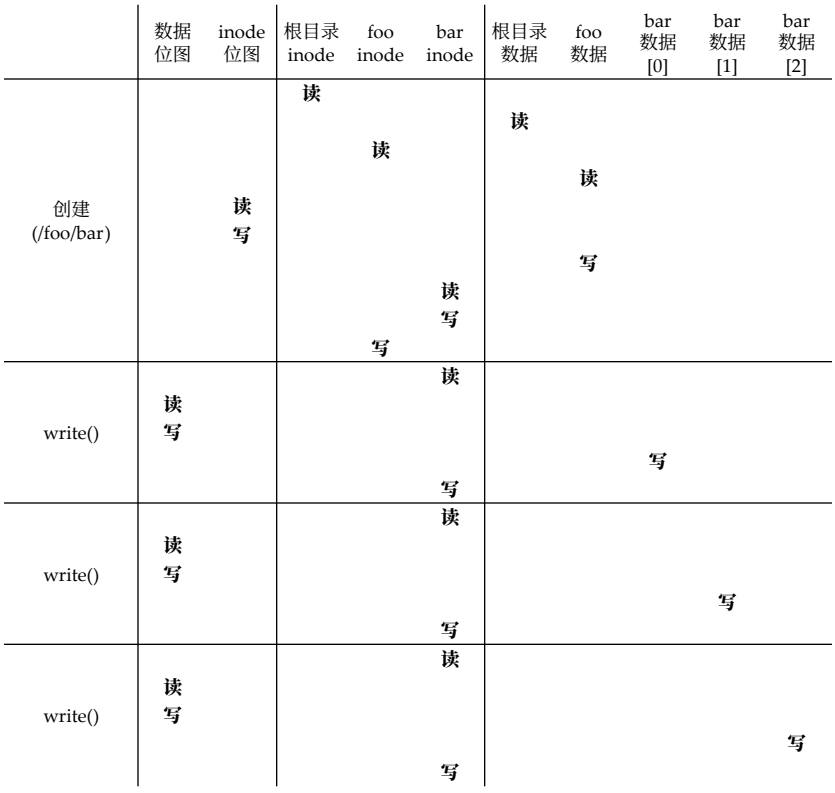


图 40.4：**创建文件的时间线（时间向下递增）**

写一次新 inode 本身（初始化它），写一次目录的数据（把文件的 high-level name 链接到它的 inode 号），再读写一次目录的 inode 来更新它。如果目录需要增长才能容纳新条目，还需要额外的 I/O，即访问数据位图（data bitmap）和新的目录块。仅仅创建一个文件就要做这么多事！

我们来看一个具体的例子：创建文件 `/foo/bar`，并向它写入三个块。图 40.4（第 11 页）展示了 `open()`（它创建了这个文件）和三次 4 KB 写操作期间各发生了什么。

图中对磁盘的读和写按引发它们的系统调用分组，大致按从上到下的顺序发生。你可以看到创建文件要做多少工作：在这个例子中要 10 次 I/O，先遍历路径名，最后才创建文件[2]。你还可以看到，每次需要分配的写操作要花 5 次 I/O：一对用来读取和更新 inode，另一对用来读取和更新数据位图（data bitmap），最后是写数据本身。文件系统怎样才能以合理的效率完成这些工作呢？

关键问题：如何降低文件系统的 I/O 开销
即使是打开、读或写文件这样最简单的操作，也会引发大量分散在磁盘各处的 I/O 操作。文件系统能做些什么，来降低执行这么多 I/O 的高昂开销？

40.7 缓存和缓冲

正如上面的例子所示，读写文件可能代价很高，会对（缓慢的）磁盘产生许多 I/O。这显然是一个严重的性能问题。为了解决它，大多数文件系统都会充分利用系统内存（DRAM）来缓存（cache）重要的块。

设想一下上面那个 open 的例子：如果没有缓存，每打开一个文件，目录层次结构（directory hierarchy）中的每一级都至少需要读两次（一次读相关目录的 inode，至少一次读它的数据）。如果路径名很长（比如 `/1/2/3/ ... /100/file.txt`），文件系统仅仅为了打开文件，就真的要执行几百次读！

因此，早期的文件系统引入了一个**固定大小的缓存**（fixed-size cache）来存放常用的块。和我们讨论虚拟内存时一样，由 **最近最少使用**（Least-Recently-Used，LRU）及其各种变体之类的策略决定哪些块保留在缓存中。这个固定大小的缓存通常在启动时分配，大约占总内存的 10%。

然而，这种对内存的**静态划分**（static partitioning）可能造成浪费；如果文件系统在某个时刻并不需要 10% 的内存呢？采用上面描述的固定大小的方法，文件缓存（file cache）中没有用到的页（page）无法挪作他用，就被浪费了。

相比之下，现代系统采用**动态划分**（dynamic partitioning）的方法。具体来说，许多现代操作系统把虚拟内存页和文件系统页整合到一个**统一的页缓存**（unified page cache）中 [S00]。这样，就能看某一时刻虚拟内存和文件系统谁更需要，在两者之间更灵活地分配内存。

[2]这里有一点出人意料：创建 bar 的 inode 时有一次读，尽管它还没有初始化；你能猜到为什么吗？（想一想……）答案：之所以有这次读，是因为 inode（它很小）包含在一个更大的块（比如 4 KB）中。因此，系统要执行一次读-改-写，来更新块中新分配的 inode。

提示：理解静态划分与动态划分

在不同的客户或用户之间分配一种资源时，可以使用**静态划分**（static partitioning），也可以使用**动态划分**（dynamic partitioning）。静态方法只是一次性地把资源按固定比例分好；例如，如果内存有两个可能的使用者，你可以把固定比例的内存分给其中一个，其余的分给另一个。动态方法更灵活，随着时间的推移分出不同数量的资源；例如，某个用户可能在一段时间内得到较高比例的磁盘带宽（bandwidth），但之后系统可能会改变主意，把可用磁盘带宽的更大份额分给另一个用户。

两种方法各有优点。静态划分（static partitioning）确保每个用户都能得到一定份额的资源，性能通常更可预测，而且往往更容易实现。动态划分（dynamic partitioning）可以实现更高的利用率（utilization），让渴求资源的用户使用原本闲置的资源，但实现起来可能更复杂；而且，如果某个用户闲置的资源被别人占用，需要时又要花很长时间才能收回，这个用户的性能就可能更差。和往常一样，没有最好的方法；你应该思考手头的问题，决定哪种方法最合适。说真的，你难道不应该一直这么做吗？

现在设想有缓存时打开文件的例子。第一次打开可能会产生大量 I/O 流量，读入目录的 inode 和数据，但之后再打开同一个文件（或者同一目录下的文件）时，大多会命中缓存，因此不需要 I/O。

我们再来看看缓存对写的影响。只要缓存足够大，读 I/O 就可以完全避免，但写流量必须到达磁盘，才能持久保存。因此，缓存对写流量起不到对读那样的过滤作用。话虽如此，人们有时所说的**写缓冲**（write buffering）无疑有不少性能上的好处。首先，推迟写入，文件系统就可以把一些更新**批量**（batch）合并成更少的 I/O；例如，如果创建一个文件时更新了 inode 位图，片刻之后创建另一个文件时又更新了它，那么第一次更新之后推迟写入，文件系统就省掉了一次 I/O。其次，把一些写缓冲在内存中，系统随后就可以**调度**（schedule）这些 I/O，从而提高性能。最后，有些写推迟之后，就可以完全**避免**（avoid）；例如，如果应用程序创建了一个文件，然后又删除了它，推迟那些把文件创建反映到磁盘上的写，就能完全避免它们。在这种情况下，（在把块写到磁盘这件事上）懒惰是一种美德。

出于上述原因，大多数现代文件系统会在内存中缓冲写操作 5 到 30 秒，这又是一种权衡：如果系统在更新传播到磁盘之前崩溃，这些更新就会丢失；然而，把写在内存中多留一段时间，就能批量处理、调度，甚至干脆省掉一些写，从而提高性能。

有些应用程序（比如数据库）并不喜欢这种权衡。因此，为了避免写缓冲（write buffering）导致意外的数据丢失，它们干脆强制把数据写到磁盘上：调用 `fsync()`，使用绕过缓存的**直接 I/O**（direct I/O）接口，或者使用**裸设备**（raw disk）接口、完全绕开文件系统[3]。虽然大多数应用程序都接受文件系统所做的权衡，但如果默认行为不能令人满意，也有足够的控制手段让系统按你的意愿行事。

[3]去上一门数据库课，就能更多地了解老派的数据库，以及它们过去坚持绕开操作系统、一切都由自己掌控的做法。不过要小心！搞数据库的那帮人总想说操作系统的坏话。真丢人，搞数据库的人。真丢人。

提示：理解持久性与性能的权衡

存储系统常常让用户面对持久性（durability）与性能之间的权衡。如果用户希望写入的数据立即持久，系统就必须费尽力气把新写入的数据提交到磁盘，因此写操作很慢（但安全）。然而，如果用户能容忍丢失一点数据，系统就可以把写操作在内存中缓冲一段时间，稍后再（在后台）写入磁盘。这样做会让写操作看起来很快就完成了，从而提高了用户感受到的性能；不过，一旦发生崩溃，还没提交到磁盘的写就会丢失，这就是其中的权衡。要正确地做出这种权衡，最好先弄清使用这个存储系统的应用程序需要什么；例如，丢失浏览器最后下载的几张图片也许还能容忍，但丢失一笔往你银行账户里存钱的数据库事务（transaction）的一部分，可能就不那么能容忍了。当然，除非你很有钱；那样的话，你干吗还这么在乎攒下每一分钱呢?

40.8 小结

我们已经看到了构建文件系统所需的基本机制。每个文件都需要一些信息，即元数据，通常存放在一种叫作 inode 的结构中。目录只是一种特殊类型的文件，存放着从名字到 inode 号的映射。还需要其他结构；例如，文件系统通常使用位图之类的结构来记录哪些 inode 或数据块是空闲的、哪些已分配。

文件系统设计最棒的一点是它的自由；我们在后面几章中探讨的文件系统，都利用这种自由优化了文件系统的某个方面。显然，还有许多策略上的决策我们没有探讨。例如，创建一个新文件时，应该把它放在磁盘的什么位置?这个策略以及其他一些策略也将是后面几章的主题。真的会吗？[4]

[4]此处响起神秘的音乐，让你对文件系统这个话题更加好奇。

 © 2008–26, ARPACI-DUSSEAU

参考文献

[A+07] "A Five-Year Study of File-System Metadata" by Nitin Agrawal, William J. Bolosky, John R. Douceur, Jacob R. Lorch. FAST '07, San Jose, California, February 2007. *An excellent recent analysis of how file systems are actually used. Use the bibliography within to follow the trail of file-system analysis papers back to the early 1980s.*

[B07] "ZFS: The Last Word in File Systems" by Jeff Bonwick and Bill Moore. Available from: http://www.ostep.org/Citations/zfs_last.pdf. *One of the most recent important file systems, full of features and awesomeness. We should have a chapter on it, and perhaps soon will.*

[B02] "The FAT File System" by Andries Brouwer. September, 2002. Available online at: http://www.win.tue.nl/%7eaeb/linux/fs/fat/fat.html. *A nice clean description of FAT. The file system kind, not the bacon kind. Though you have to admit, bacon fat probably tastes better.*

[C94] "Inside the Windows NT File System" by Helen Custer. Microsoft Press, 1994. *A short book about NTFS; there are probably ones with more technical details elsewhere.*

[H+88] "Scale and Performance in a Distributed File System" by John H. Howard, Michael L. Kazar, Sherri G. Menees, David A. Nichols, M. Satyanarayanan, Robert N. Sidebotham, Michael J. West.. ACM TOCS, Volume 6:1, February 1988. *A classic distributed file system; we'll be learning more about it later, don't worry.*

[P09] "The Second Extended File System: Internal Layout" by Dave Poirier. 2009. Available: http://www.nongnu.org/ext2-doc/ext2.html. *Some details on ext2, a very simple Linux file system based on FFS, the Berkeley Fast File System. We'll be reading about it in the next chapter.*

[RT74] "The UNIX Time-Sharing System" by Dennis M. Ritchie, Ken Thompson. CACM Volume 17:7, 1974. *The original paper about* UNIX. *Read it to see the underpinnings of much of modern operating systems. It's amazing how many things they simply got right!*

[S00] "UBC: An Efficient Unified I/O and Memory Caching Subsystem for NetBSD" by Chuck Silvers. FREENIX, 2000. *A nice paper about NetBSD's integration of file-system buffer caching and the virtual-memory page cache. Many other systems do the same type of thing.*

[S+96] "Scalability in the XFS File System" by Adan Sweeney, Doug Doucette, Wei Hu, Curtis Anderson, Mike Nishimoto, Geoff Peck. USENIX '96, January 1996, San Diego, California. *The first attempt to make scalability of operations, including things like having millions of files in a directory, a central focus. A great example of pushing an idea to the extreme. The key idea behind this file system: everything is a tree. We should have a chapter on this file system too.*

作业（模拟）

用 `vsfs.py` 这个工具研究文件系统的状态如何随各种操作而变化。文件系统一开始处于空的状态，只有一个根目录。随着模拟的进行，会执行各种操作，从而慢慢改变文件系统在磁盘上的状态。详见 README。

问题

1. 用几个不同的随机种子（比如 17、18、19、20）运行模拟器，看看你能不能推断出每两次状态变化之间一定发生了哪些操作。
2. 现在用不同的随机种子（比如 21、22、23、24）做同样的事，只是加上 `-r` 选项运行，这样会显示出操作，让你去推测状态的变化。关于 inode 和数据块的分配算法，你能得出什么结论？它们倾向于分配哪些块？
3. 现在把文件系统中的数据块数量减到很少（比如两个），运行模拟器，处理一百个左右的请求。在这种限制非常严格的布局下，文件系统里最终会有哪些类型的文件？哪些类型的操作会失败？
4. 现在对 inode 做同样的事。inode 很少时，哪些类型的操作能成功？哪些通常会失败？文件系统最终的状态可能是什么样子？

 © 2008–26, ARPACI-DUSSEAU